

Apache Iceberg

Core 模块源码设计文档

版本: 1.9.2 | 核心流程图 / 时序图 / 类设计说明

Apache Iceberg 是高性能的开放表格式(Open Table Format)，专为超大规模分析型数据集设计。Core 模块是核心引擎，提供表元数据管理、快照机制、Manifest 文件组织、扫描规划、事务控制和 Catalog 抽象等关键功能。本文档基于 v1.9.2 源码深度分析。

目录

第一章 Iceberg Core 整体架构概览

- 1.1 模块分层架构
- 1.2 核心包结构
- 1.3 数据文件组织模型

第二章 核心流程图

- 2.1 数据写入提交流程
- 2.2 数据扫描规划流程
- 2.3 表元数据更新流程
- 2.4 事务提交流程
- 2.5 Catalog 加载表流程

第三章 核心时序图

- 3.1 FastAppend 写入时序
- 3.2 DataTableScan 扫描时序
- 3.3 Transaction 多操作提交时序
- 3.4 REST Catalog 表操作时序

第四章 核心类设计说明

- 4.1 TableMetadata
- 4.2 SnapshotProducer 体系
- 4.3 ManifestGroup 扫描引擎
- 4.4 DeleteFileIndex
- 4.5 BaseTransaction 事务机制
- 4.6 Catalog 体系
- 4.7 FileIO/Writer 体系

第五章 关键设计模式总结

第一章 Iceberg Core 整体架构概览

1.1 模块分层架构

Iceberg Core 模块采用清晰的分层架构设计，从上到下分为 API 层、核心实现层、存储抽象层和序列化层。Core 模块是引擎无关的(Engine-agnostic)，不依赖任何特定计算引擎(Spark/Flink)。

分层	核心职责	关键类/包
API 层 (api/)	定义公共接口和类型	Table, Catalog, Schema, PartitionSpec, Scan 等接口
核心实现层 (core/)	表规范的完整实现	TableMetadata, SnapshotProducer, ManifestGroup, BaseTransaction
Catalog 层	Catalog 服务的多种实现	RESTSessionCatalog, HadoopCatalog, JdbcCatalog, CachingCatalog
存储抽象层 (core/io/)	统一的文件 IO 接口和写入框架	FileIO, HadoopFileIO, ResolvingFileIO, BaseTaskWriter
序列化层 (core/avro/)	Avro 格式读写，元数据序列化	Avro, ManifestWriter, ManifestReader, TableMetadataParser

1.2 核心包结构

包名	文件数	核心功能
org.apache.iceberg (根包)	~185	表元数据、快照、Manifest、扫描、写操作、事务等核心实现
org.apache.iceberg.rest	~92	REST Catalog 完整实现，含 OAuth2、请求/响应模型
org.apache.iceberg.avro	~44	Avro 格式读写核心，Schema 转换，值编解码
org.apache.iceberg.io	~41	IO 抽象层、任务写入器、滚动写入器、分区写入器
org.apache.iceberg.util	~35	工具类集合：Tasks 任务框架、快照工具、线程池等
org.apache.iceberg.hadoop	~14	Hadoop 生态集成：FileIO、Catalog、TableOperations
org.apache.iceberg.jdbc	~7	JDBC Catalog 实现

1.3 数据文件组织模型

Iceberg 采用三层文件组织模型：Metadata File -> Manifest List -> Manifest File -> Data/Delete File。每个层级都通过快照(Snapshot)机制实现 MVCC，支持时间旅行和增量读取。

Iceberg 数据文件组织层次模型

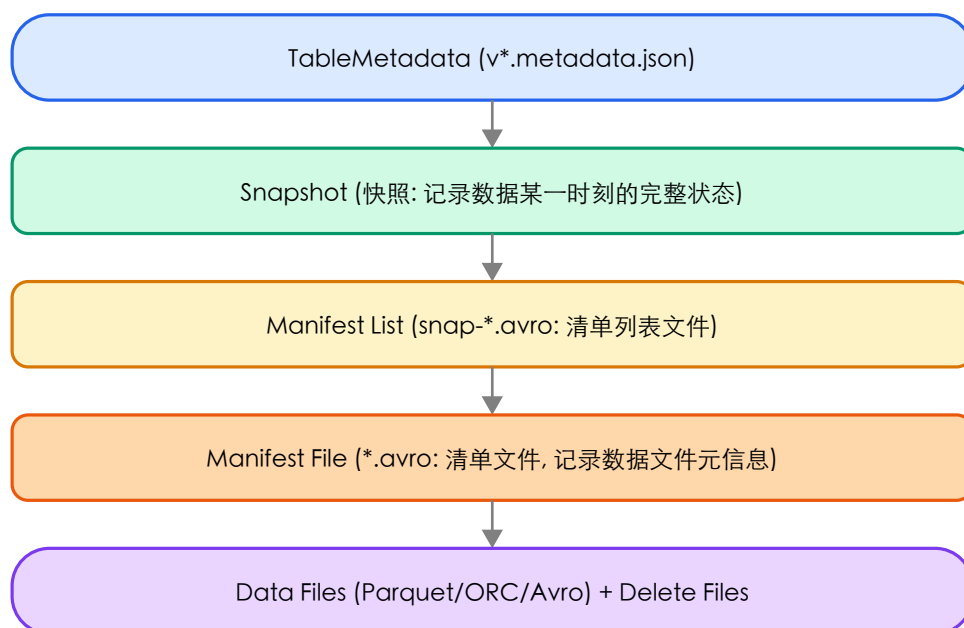


图 1-1: Iceberg 数据文件的层次组织模型

核心设计理念：

1. 不可变性：TableMetadata/Schema/PartitionSpec 均为不可变对象，通过 Builder 产生新实例
2. MVCC：每次写操作产生新快照，读写不互相阻塞
3. 乐观并发：写操作使用 CAS(Compare-And-Swap) 提交
4. Schema Evolution：通过 field ID 而非列名关联数据

第二章 核心流程图

2.1 数据写入提交流程 (Snapshot Commit)

`SnapshotProducer` 是所有写操作的基类，其 `commit()` 方法实现了带重试的乐观并发提交机制。流程包括：生成快照、写入 Manifest、提交元数据更新、冲突检测与重试、清理未提交文件。

SnapshotProducer.commit() 数据写入提交流程

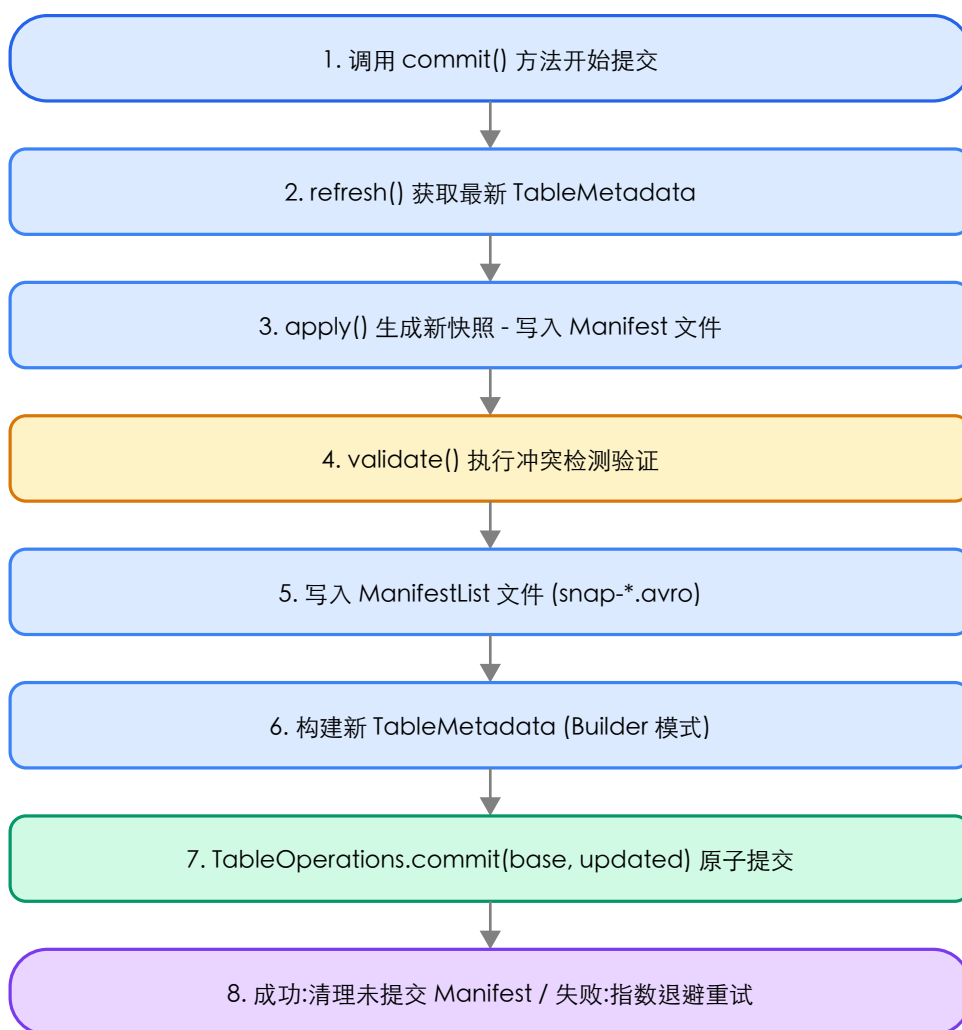


图 2-1: SnapshotProducer.commit() 完整流程

关键机制：乐观重试使用 `Tasks.foreach()` 框架，默认最多重试 4 次指数退避；`validate()` 在 `apply()` 前调用检查冲突；提交后 `cleanUncommitted()` 删除临时 Manifest。

2.2 数据扫描规划流程 (Scan Planning)

`DataTableScan` 是最常用的扫描入口，委托 `ManifestGroup` 进行 `Manifest` 过滤和文件规划。支持三级过滤：Manifest 级、分区级和文件级。

DataTableScan 扫描规划流程

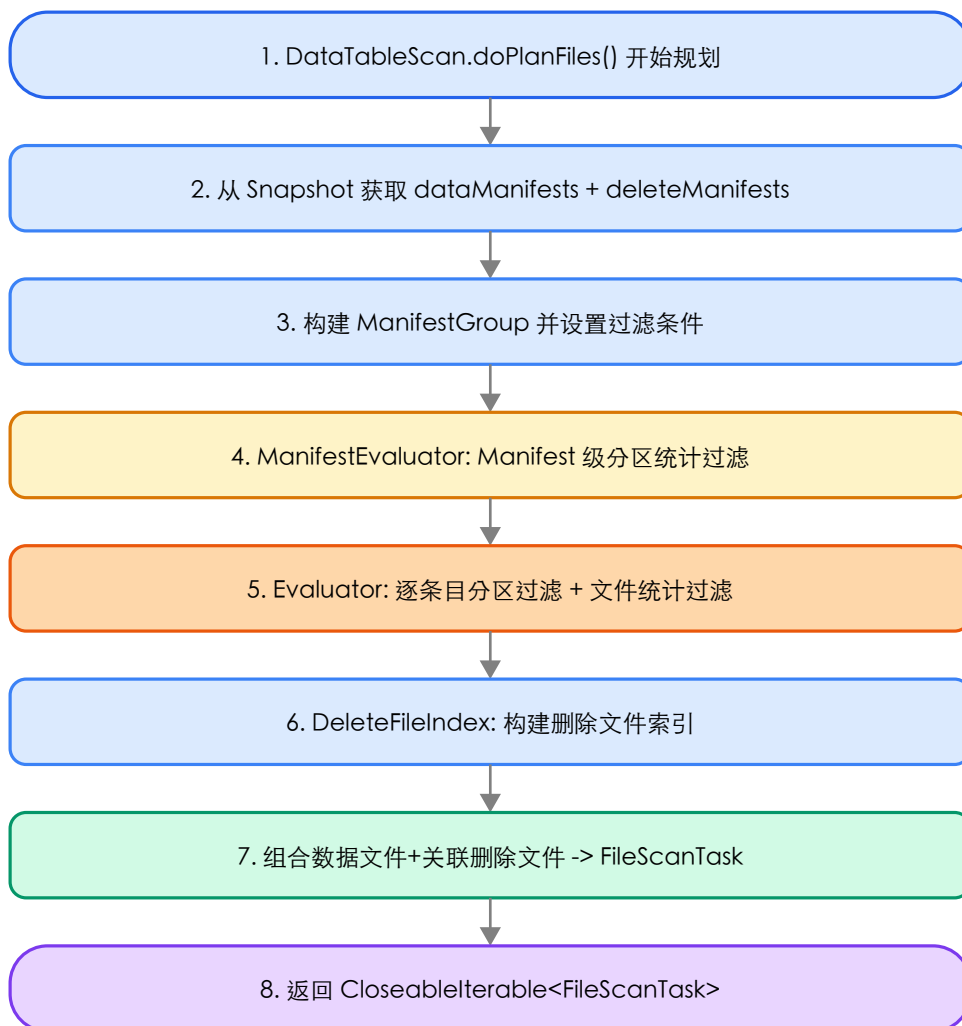


图 2-2: `DataTableScan` 数据扫描规划完整流程

2.3 表元数据更新流程

TableMetadata 更新流程

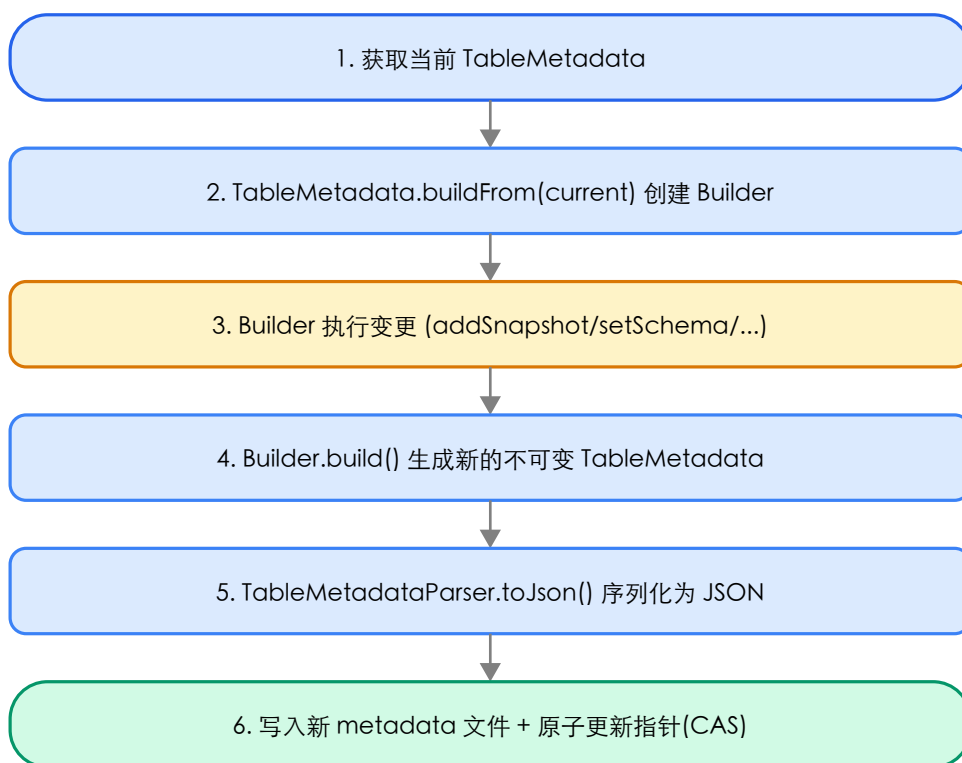


图 2-3: 表元数据更新流程

2.4 事务提交流程

Transaction 事务提交流程

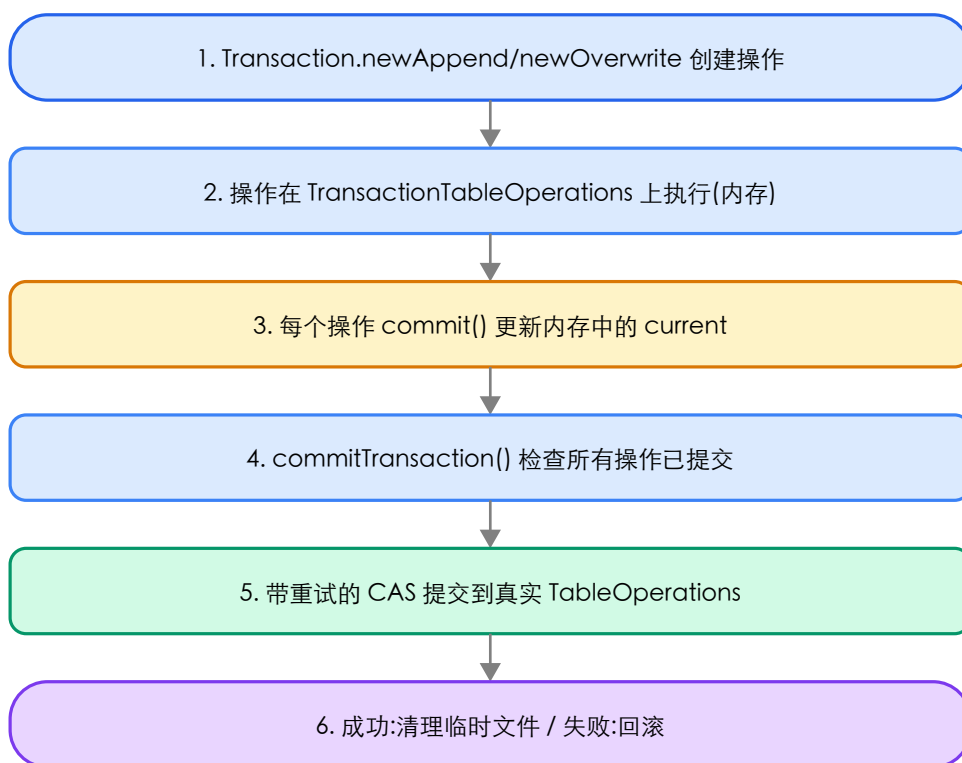


图 2-4: 事务提交流程

2.5 Catalog 加载表流程

Catalog 加载表流程

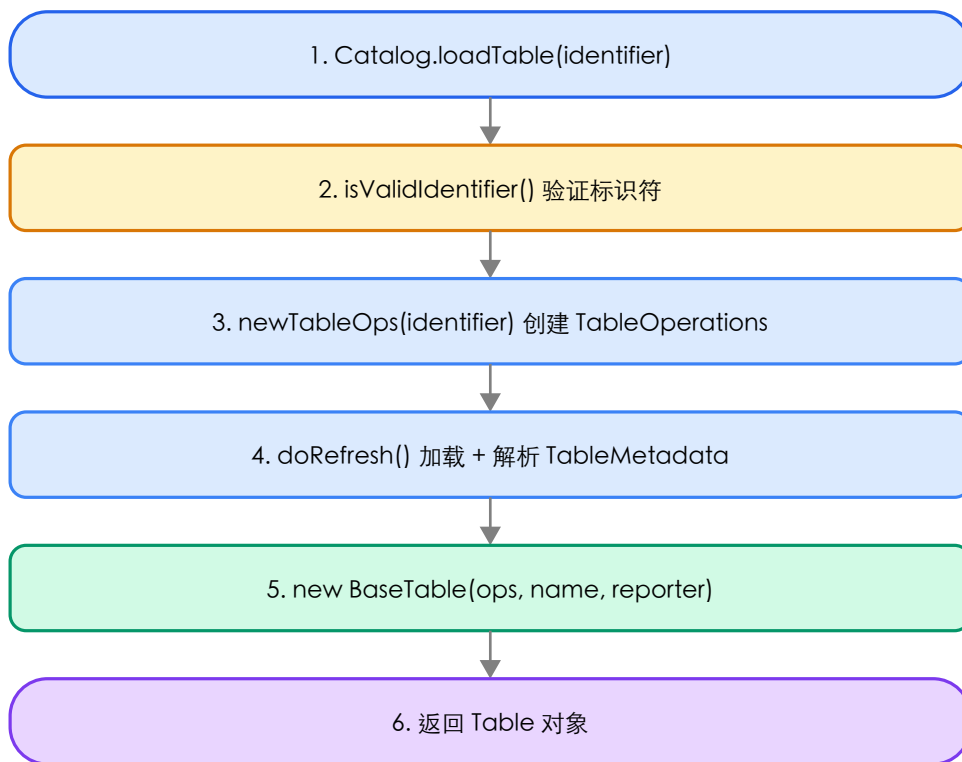


图 2-5: Catalog 加载表完整流程

第三章 核心时序图

3.1 FastAppend 写入时序

FastAppend 直接追加新 Manifest 文件，不合并已有 Manifest，适用于流式写入和小批量追加。

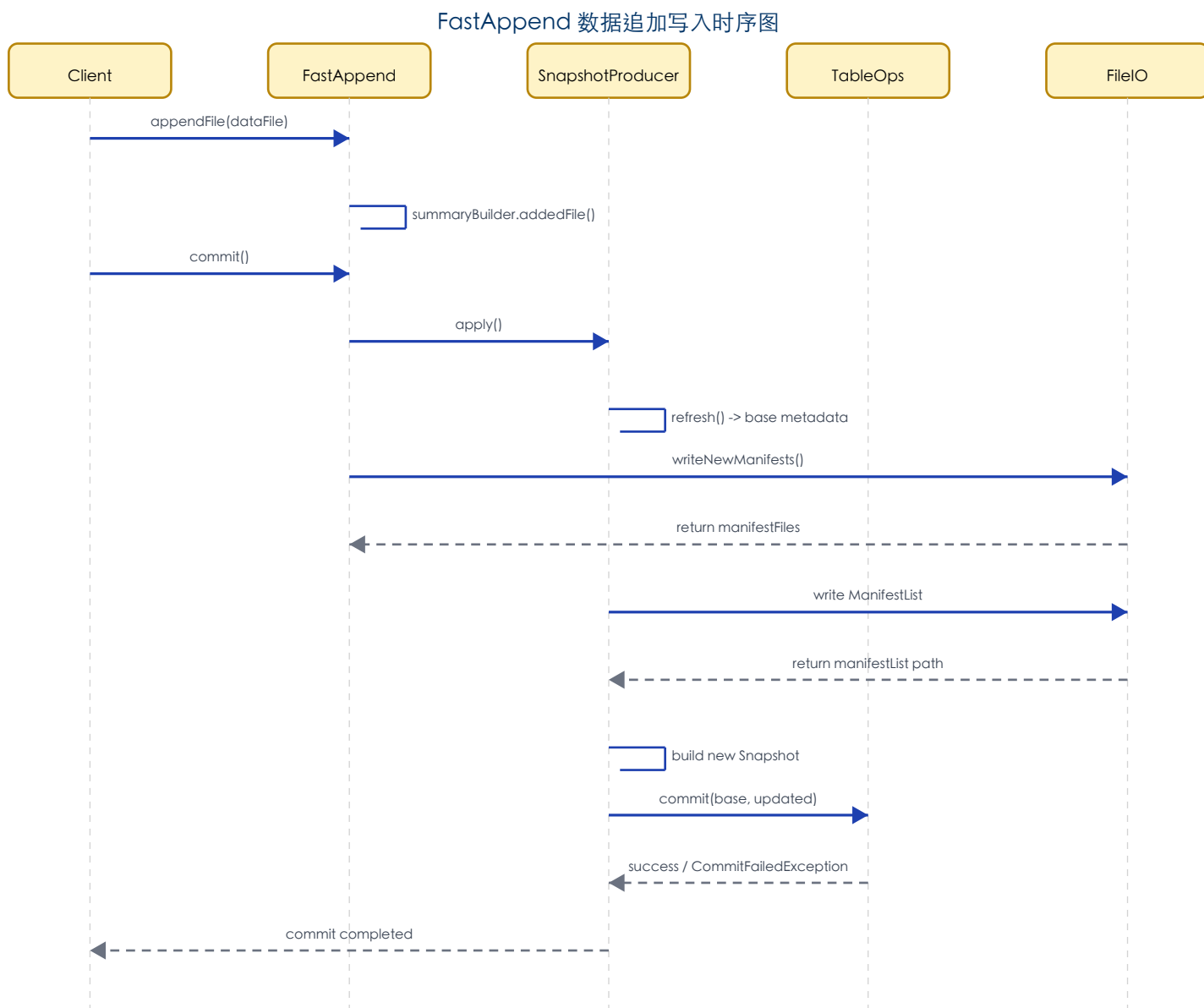


图 3-1: FastAppend 数据追加写入时序图

3.2 DataTableScan 扫描时序

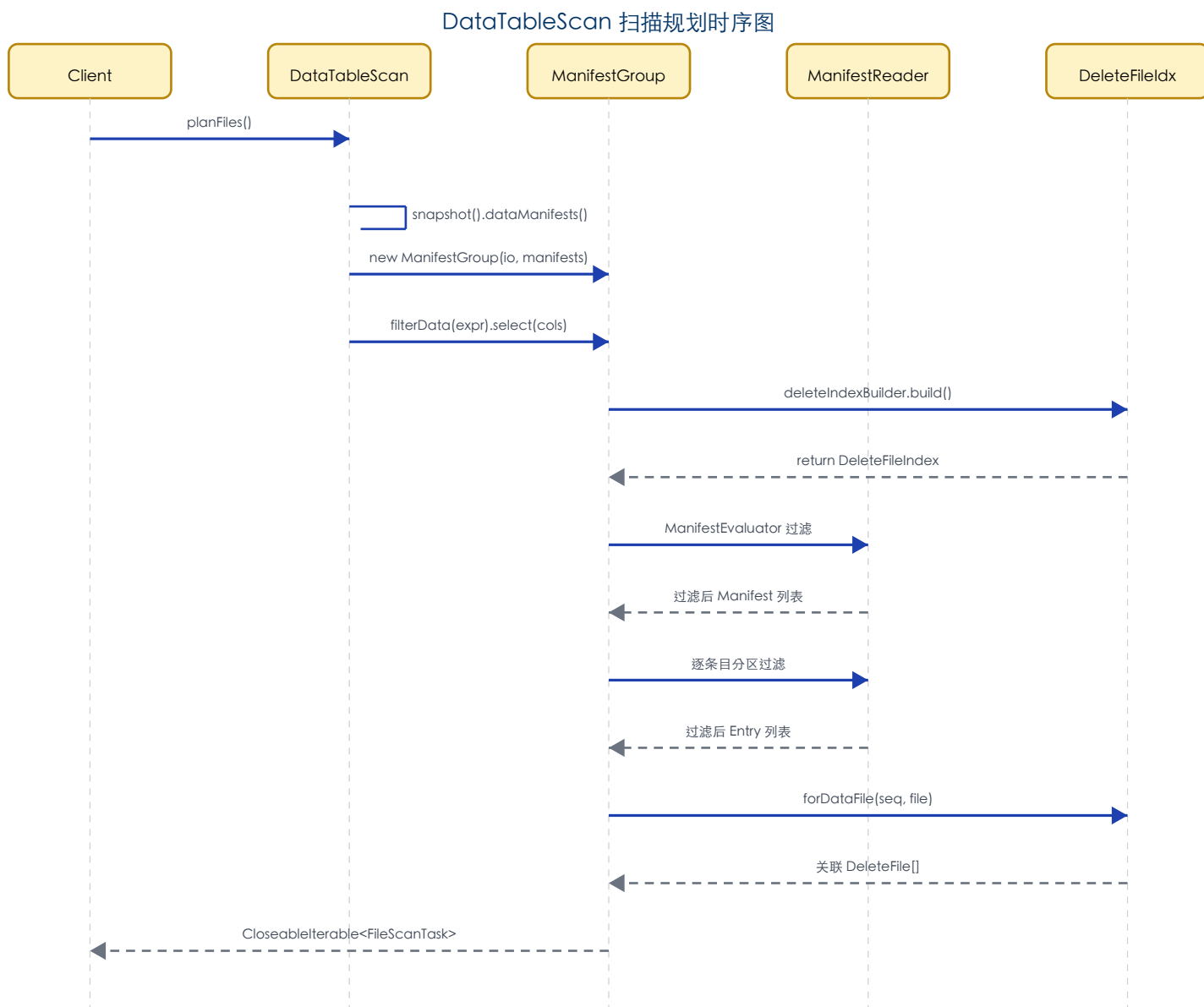


图 3-2: DataTableScan 扫描规划时序图

3.3 Transaction 多操作提交时序

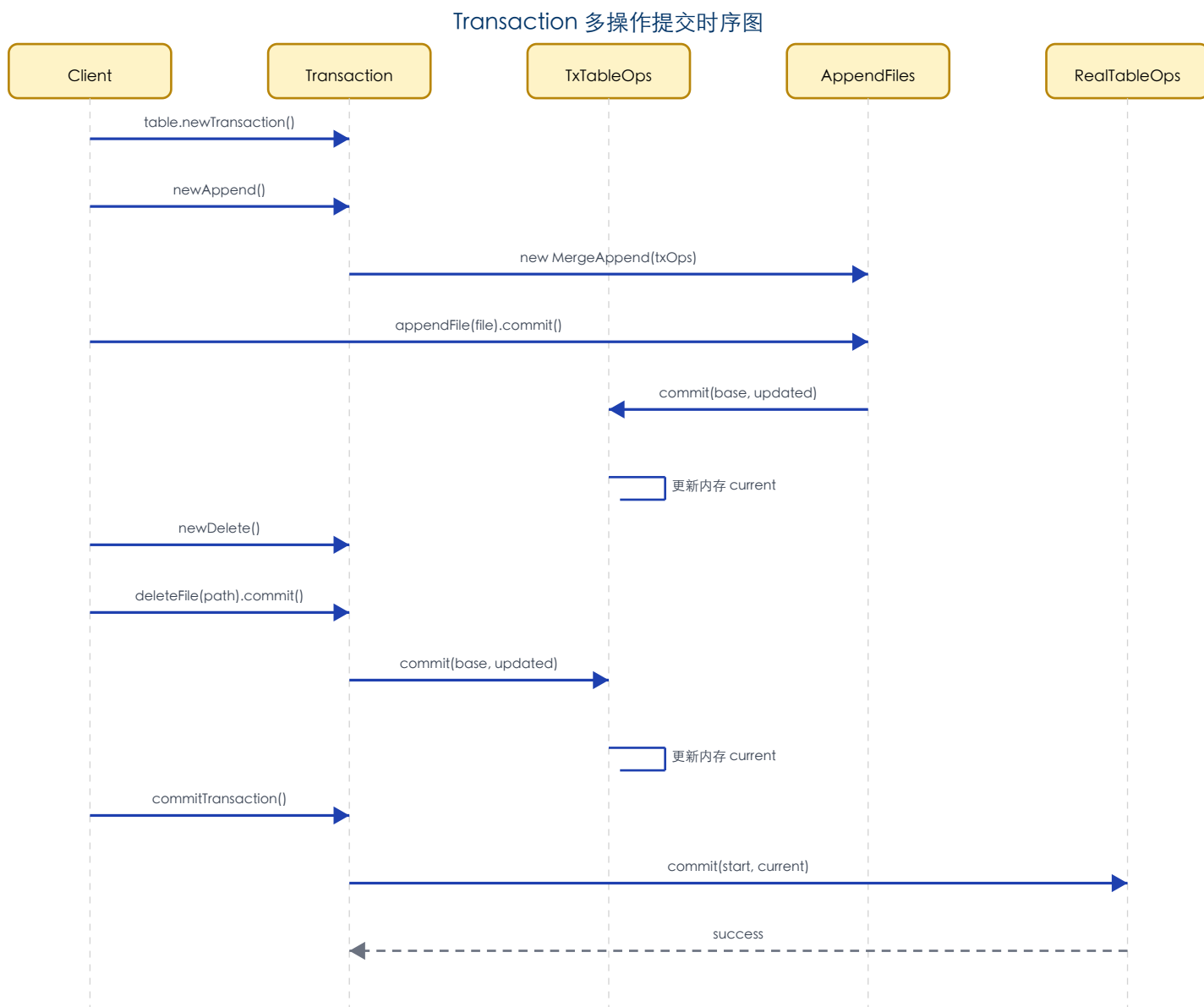


图 3-3: Transaction 多操作事务提交时序图

3.4 REST Catalog 表操作时序

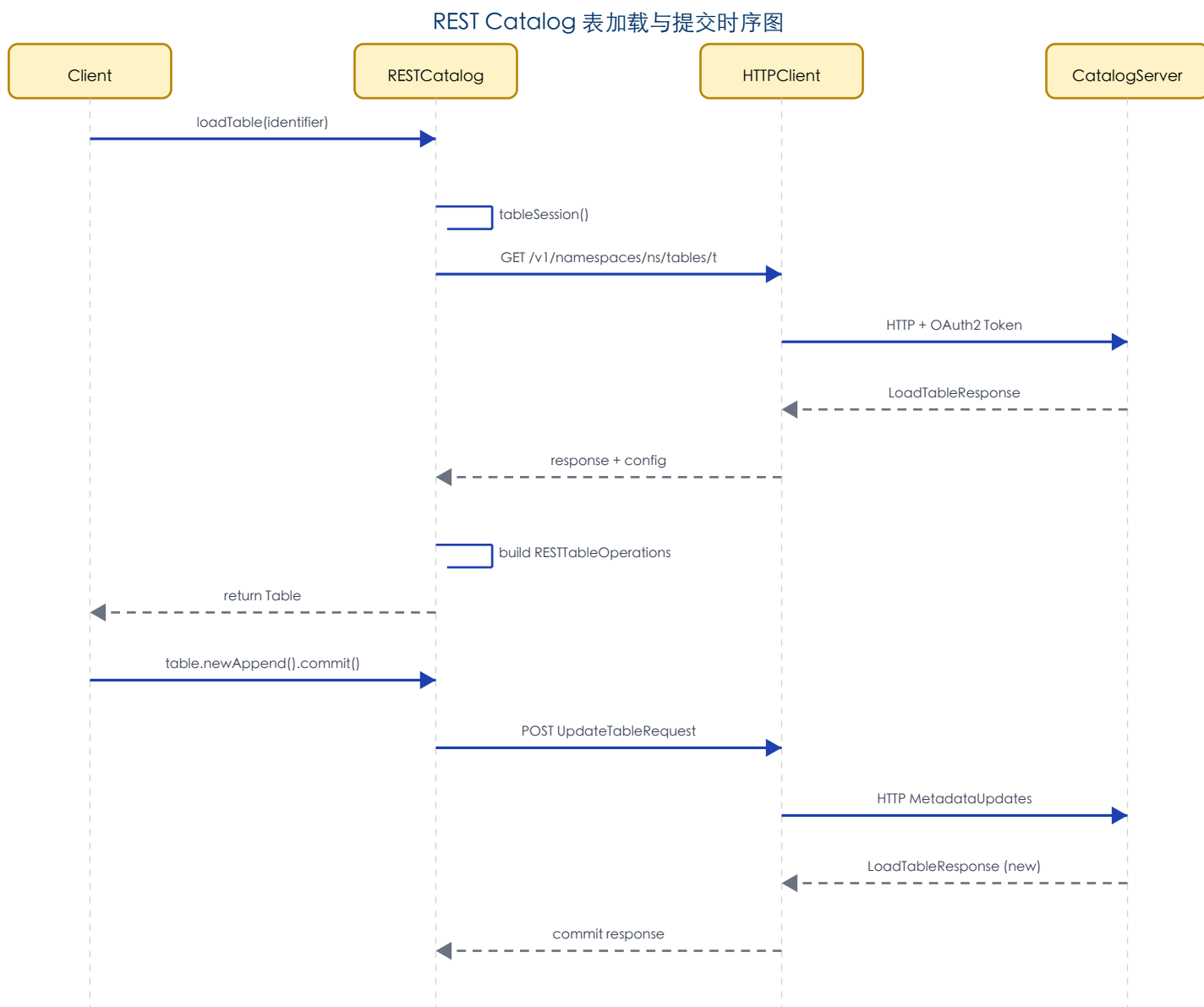


图 3-4: REST Catalog 表加载与提交操作时序图

第四章 核心类设计说明

4.1 TableMetadata - 表元数据核心

TableMetadata (67KB) 是 Iceberg 最核心的数据结构，代表表在某一时刻的完整元数据状态。不可变的对象，所有修改通过内部 Builder 产生新实例。

字段名	类型	说明
formatVersion	int	表格式版本 (1/2/3/4)，控制可用特性集
uuid	String	表的唯一标识，创建后不变
location	String	表数据的根存储路径
lastSequenceNumber	long	最后使用的序列号，用于 MVCC 可见性
schemas / currentSchemaId	List/int	历史 Schema 列表和当前 Schema ID
specs / defaultSpecId	List/int	分区规格列表和默认分区规格
properties	Map	表属性配置(压缩、分割大小等)
snapshots	List	快照列表，每个快照记录一次写操作的结果
refs	Map	分支和标签引用 (main branch + tags)

设计要点：

1. 不可变性：所有集合使用 ImmutableList/ImmutableMap
2. 版本兼容：formatVersion 控制特性门控，V2+ Row-level Delete，V3+ Row Lineage
3. 增量更新：Builder 内部维护 changes 列表，记录 MetadataUpdate 事件

4.2 SnapshotProducer - 快照生产者体系

SnapshotProducer 是所有数据写入操作的抽象基类，MergingSnapshotProducer 增加了 Manifest 合并和冲突检测。

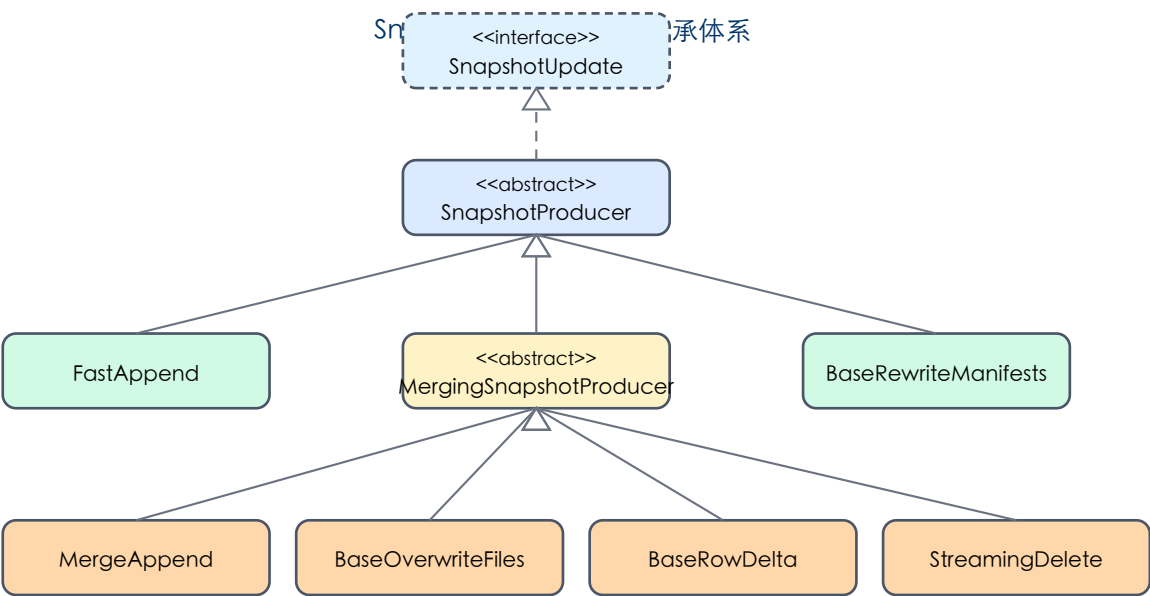


图 4-1: SnapshotProducer 类继承体系

操作类	operation()	特点
FastAppend	append	直接追加 Manifest，不合并，性能最优
MergeAppend	append	合并小 Manifest，适合频繁小写入
BaseOverwriteFiles	overwrite	按条件覆写，需冲突检测
BaseRowDelta	overwrite	同时添加数据文件和删除文件
BaseReplacePartitions	overwrite	动态覆写指定分区的数据
StreamingDelete	delete	流式删除数据文件
BaseRewriteFiles	replace	文件重写(compact)

4.3 ManifestGroup - 扫描引擎

ManifestGroup (17KB) 是扫描规划的核心引擎，实现三级过滤机制，支持并行扫描。

过滤层级	实现类	过滤依据
第一级: Manifest 过滤	ManifestEvaluator	利用 Manifest 分区统计(min/max)跳过整个 Manifest
第二级: 分区过滤	Evaluator	利用条目分区值判断是否匹配查询条件
第三级: 文件统计过滤	InclusiveMetricsEvaluator	利用文件级 column min/max/null-count 过滤

4.4 DeleteFileIndex - 删除文件索引

DeleteFileIndex (32KB) 是 V2 格式的关键组件，支持三种删除：等值删除、位置删除和删除向量(DV)。

索引类型	匹配策略
globalDeletes	序列号 > 数据文件序列号的等值删除应用于所有文件
eqDeletesByPartition	按分区匹配，序列号大于数据文件的等值删除才生效
posDeletesByPartition	按分区匹配，关联同一分区的位置删除文件
posDeletesByPath	按数据文件路径精确匹配位置删除文件
dvByPath	按数据文件路径精确匹配 Deletion Vector

4.5 BaseTransaction - 事务机制

BaseTransaction (22KB) 实现多操作事务语义，支持四种事务类型。

事务类型	使用场景	提交方式
CREATE_TABLE	创建新表	直接 commit(null, metadata)，不可重试
REPLACE_TABLE	替换已有表	带重试的 CAS commit
CREATE_OR_REPLACE_TABLE	创建或替换表	先替换，失败后创建
SIMPLE	普通多操作事务	带重试 CAS，冲突时重新应用操作

4.6 Catalog 体系 - 目录服务抽象

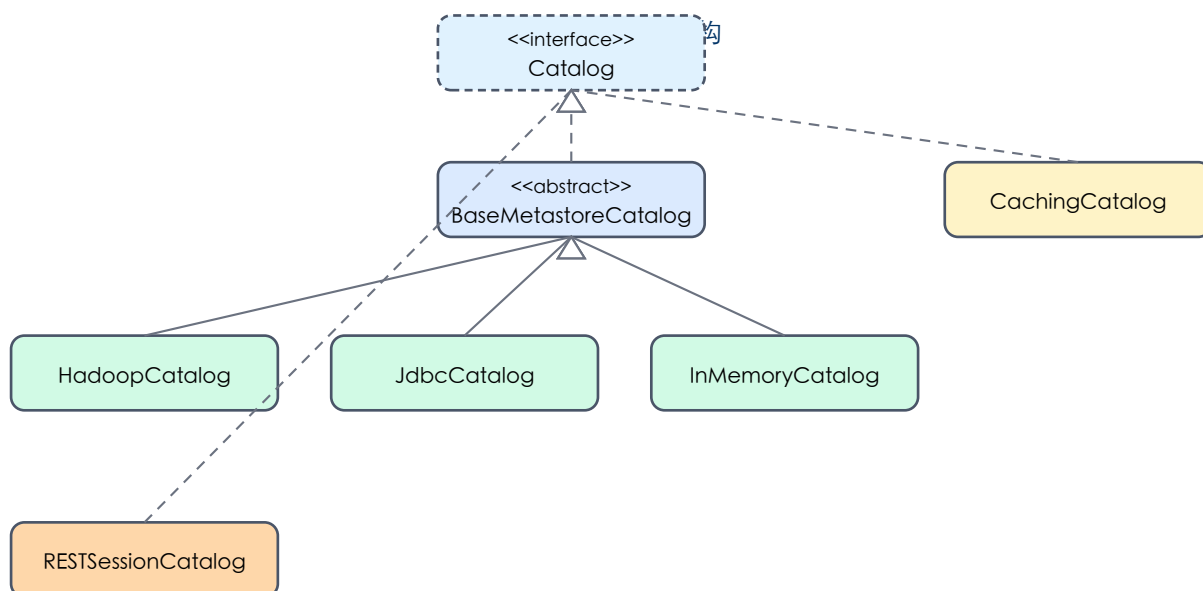


图 4-2: Catalog 类体系结构

Catalog	存储介质	适用场景
HadoopCatalog	HDFS/S3 文件系统	简单部署，无外部依赖

JdbcCatalog	RDBMS (MySQL/PG)	已有 RDBMS 环境
InMemoryCatalog	内存	单元测试专用
RESTSessionCatalog	REST API	生产级多租户环境，支持 OAuth2 和服务端规划
CachingCatalog	(装饰器)	为任意 Catalog 添加内存缓存层

4.7 FileIO 与 Writer 体系

组件	说明
HadoopFileIO	基于 Hadoop FileSystem API，支持 HDFS 和兼容 S3A
InMemoryFileIO	内存实现，用于测试
ResolvingFileIO	根据 URI scheme 自动委托到对应 FileIO (s3://->S3FileIO)
BaseTaskWriter	任务写入器基类(17KB)，管理打开的文件写入器
ClusteredWriter	要求输入按分区排序，一次只打开一个分区写入器
FanoutDataWriter	同时打开多个分区写入器，适合无序数据
RollingFileWriter	单文件超大小限制时自动滚动到新文件
OutputFileFactory	生成带分区目录结构的输出文件路径

第五章 关键设计模式总结

1. 不可变对象 + Builder 模式

TableMetadata/Schema/PartitionSpec 均为不可变对象，所有修改通过 Builder 产生新实例。天然线程安全，Builder 内部跟踪 MetadataUpdate 变更事件。

2. 乐观并发控制 (OCC)

所有写操作通过 CAS 机制实现乐观并发。使用 Tasks.foreach() 框架提供指数退避重试(默认 4 次)。失败后重新读取 base 并重新 apply。

3. 模板方法模式

SnapshotProducer 定义 commit 骨架，子类只需实现 apply()/operation()/summary() 等抽象方法。

4. Refinement 模式

TableScan 的 filter()/select() 等返回新的独立扫描对象，状态不会泄漏。确保扫描对象的不可变性和线程安全。

5. 三级过滤优化

ManifestGroup 实现 Manifest 级->分区级->文件统计级三级过滤，层层递进减少扫描数据量。

6. 装饰器模式

CachingCatalog 添加缓存层；EncryptingFileIO 添加加密；ResolvingFileIO 动态委托。提供灵活的功能组合。

7. CloseableIterable 统一抽象

使用 CloseableIterable 而非 Stream 作为惰性集合抽象。确保资源正确关闭(ManifestReader 文件句柄)。

8. 序列号驱动的 MVCC

每个快照分配递增的序列号，删除文件只对序列号更小的数据文件生效。保证读取一致性。

9. 自定义序列化

不使用 Jackson 注解，采用自定义 XxxParser.toJson/fromJson。JSON 键名 kebab-case。

总结：Iceberg Core 模块约 500 个 Java 源文件，通过精心设计的分层架构、不可变对象、乐观并发、三级过滤等机制，实现了高性能、高可靠的开放表格式。其核心类 TableMetadata(67KB)、MergingSnapshotProducer(49KB)、SnapshotProducer(33KB)、DeleteFileIndex(32KB)、RESTSessionCatalog(64KB) 共同构成了引擎无关的数据湖基础设施。